DREW

Lecture 26: Linux Commands

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

11/01/2024

HW for you over the weekend!!

- C Programming Language
  - **CLion** by JetBrains: free for education, register with your Drew email.
  - https://www.jetbrains.com/clion/
    - **Mac**: If already have **Xcode**, update it latest version.
      Otherwise, install Xcode via App Store.
    - **Windows**: Install **gcc** via **Cygwin**.
    - https://www.cygwin.com/
    - **Tutorial** https://www.youtube.com/watch?v=WmCwxKHLNH4

Overview

- What is Linux? An open-source operating system.
- Why Linux? Security, privacy, customizations, portable, ubiquitous; libraries, software packages, and management; powerful tools in the command line.
- Popular distributions of Linux: Ubuntu, CentOS, Red Hat, Kali (for security/penetration tests) etc.
- We are using an online version: https://cocalc.com/ → Projects

DREW

Overview (Optinal)

- Why the online version?
- If you are interested in trying out, below are the steps:
  1. Download a *Virtual Machine (VM)* terminal:
     - VirtualBox (Oracle)
     - VMware Workstation Player for Windows
     - VMware Fusion Player for Macs
  2. Download the Linux system .iso.
     - Ubuntu

Overview (Optional)

- Why the online version?
- If you are interested in trying out, below are the steps:

3. Mount the image onto the VM and install the system.
   - Will be asked for spec. allocated to the VM.
   - There's required minimum spec., but can always re-spec. later.

- What you'll get is a system with GUI (graphics user interface).
- That's not the point. The command line is where the fun begins.

Let's get started

- For the sake of this course: https://cocalc.com/ → Projects

1. Go to "My First Project".
2. Go to the "New" tab on the left and choose "Linux Terminal".

- This is the *Command Line/Terminal/Shell/Bash* Interface to the Linux File System.
- We operate/interact with the system by typing in commands.

- This is what we have as of now: ~$
- The $ sign indicates that the system is waiting for an input.
- So, let's give it an input …

File System Commands

- *pwd*: **P**rint **W**orking **D**irectory; i.e., current folder/directory (dir.) we are at.

```
~$ pwd
/home/user
~$
```

- Note we have a ~ (tilde) before the $ sign.
- What's before the $ sign is the current dir.

- Thus, ~ is the /home/user dir. Where ~ is the *home dir*.
- And / is the *root dir.* that is the top-level of the file system.

- Note: The *home dir.* ~ is different from the dir. that is called "home".

DREW

File System Commands

- *pwd*: **P**rint **W**orking **D**irectory; i.e., current folder/directory (dir.) we are at.

```
~$ pwd
/home/user
~$
```

- A dir. is in the form of /xxx/xxx/…/xxx/xxx, where it shows the path that leads to this current dir., the last xxx indicates the name of current dir..

- This is important, since we can stay within a dir. and perform actions on any (other/self) dir.

File System Commands

- *ls*: **Li**sting current dir.; i.e., shows what files/folders are in the current dir.

```
~$ ls
2024-11-xx-xxxxxx.term
~$
```

- *ls [dir.]*: **Li**sting target dir.; i.e., shows what files/folders are in the target dir.

```
~$ ls /                    [Listing out the root dir.]
bin cocalc data ... ...    [bin, cocalc are folders, data is a file.]

~$ ls ~                    [Listing out the home dir.]
 2024-11-xx-xxxxxx.term        [Since we are in ~, same result.]
~$
```

DREW

File System Commands

- *ls [dir.]*: **Listing** target dir.; i.e., shows what files/folders are in the target dir.

```
~$ ls /lib
lots of things          [All different types of files.]

~$ ls /data             [data is not a file, so cannot be listed.]
~$
```

[Auto-Completion]

```
~$ ls / *tap the Tab key twice
list out all the folders in / dir.

~$ /l *tap the Tab key twice
list out all the folders in / dir.
that starts with the prefix.
```

DREW

File System Commands

- *ls [dir.]*: **Listing** target dir.; i.e., shows what files/folders are in the target dir.

[Find what's in lib/; i.e., the next level dir.]

~$ ls /lib/ *tap the Tab key twice

Display all 515 possibilities? (y or n) *waiting for y or n (only 2 options)

*If n, cancels.

*If y,

lots of things

--More-- *waiting for response

*press enter, print 1 more line;

*press space, print 1 more page;

*press control+c (^C), abort.

DREW

File System Commands

- *ls [options] [dir.]*: **Li**sting target dir. with options; i.e., shows what files/folders are in the target dir with different levels of detail on the files/folders.

```
~$ ls –l                    [A long listing, permissions, time stamp, etc.]
total 1
-rw-r--r--  1  user  user  0  Oct xx  xx:xx  filename


~$ ls –l  /lib32/gconv/
total 8284
… …
```

DREW

File System Commands

- *cd [dir.]*: Access target Directory; i.e., move from the current directory to another.

```
~$ cd                       [cd back to home dir.]
~$

~$ cd /lib/gcc              [cd to the target dir.]
/lib/gcc$                   [Current working dir.]

/lib/gcc$ cd ..             [cd back to the previous level of (parent) dir.]
/lib$

/lib$ cd gcc/ x86_64-linux-gnu/10
/lib/gcc/x86_64-linux-gnu/10$ cd ../..      [cd back two level of dir.]
/lib/gcc$
```

File System Commands

- *mkdir <dir.>*: **Make** a new **Dir**. at specific location and with specific name.
- Will check for names already in usage.

```
~$ mkdir hello                    [Create a new dir. here.]
~$ ls
 2024-11-xx-xxxxxx.term  hello
~$
```

```
~$ mkdir hello/there
~$ ls hello/
there
~$
```

```
~$ mkdir hello/../hi   [Create a new dir. at target dir.]
~$ ls
2024-11-xx-xxxxxx.term  hello  hi
```

DREW

File System Commands

- *mkdir [options] <dir.>*: **Make** a new **Dir**. at specific location and name with options.

```
~$ mkdir drew/macs/csci260
mkdir: cannot create directory '... ...': No such file or directory
~$

~$ mkdir –p drew/macs/csci260          [Create dir. and its parent(s)
~$ ls drew/macs/                       if needed to.]
csci260
~$
```

File System Commands

- *rmdir <dir.>*: **Rem**ove a **Dir**. at specific location and name with options.

```
~$ rmdir drew/macs/csci260          [Delete dir. specified in the
~$ ls drew/                         last position.]
macs
~$ ls drew/macs/                    [Empty, so returning nothing.]
~$

~ rmdir drew                        [Can only remove empty folders.]
rmdir: failed to remove ‘drew’: Directory not empty
~$                                  [There’s an alternative.]
```

File System Commands

- *cp [options] <[source dir.] source> <[dest. dir.] dest.>*: **Copy** a source [dir.]/file to a destination [dir.]/file.

mkdir drew/macs/csci260

```
~$ touch test.txt                          [Create an empty file, and
~$ cp test.txt hello.c                     copy it as a new file here, and
~$ cp hello.c drew/macs/csci260/           copy it to another dir.]

~$ cp -r drew/macs/csci260 drew/macs/math220
~$ ls drew/macs                            [Recursively copying; i.e.,
csci260 math220                            including dir. and everything
~$ ls drew/macs/math220                    within.]
 hello.c
 ~$
```

DREW

File System Commands

- *mv [options] <[source dir.] source> <[dest. dir.] dest.>*: **Move** a source [dir.]/file to a destination [dir.]/file.

```
~$ mv drew/macs/csci260/hello.c drew/macs/
~$ ls drew/macs/csci260/                    [Moving hello.c from a dir.
~$ ls drew/macs/                            to another dir.]
csci260 hello.c math220

~$ mv drew/macs/hello.c drew/macs/math220/there.c
~$ ls drew/macs/math220                     [Moving hello.c from a dir.
hello.c there.c                             to another dir. and re-name
                                            it there.c.]
```

Be careful: If not re-naming, will override without warning!!

DREW

File System Commands

- *rm [options] <[dir.] file>*: **Rem**ove/delete a *file.

```
~$ rm -v hello.c
remove 'hello.c'
```

[Remove the file hello.c, where the –v option Echos the action taken place. Many other commands also have –v option.]

Recall, *rmdir* cannot remove non-empty dir.. The alternative is to use *rm –r* command.

```
~$ rmdir drew/
rmdir: failed to remove 'drew/': Directory not empty
~$ rm –r drew/
```

[Remove the dir. drew and anything within it, dir. or file.]

Be careful: No way to recover!!

Remarks

Not limited to the mkdir command.

- For *[options]*, can have multiple options at the same time; e.g.,

```
~$ mkdir –pv drew/gened/sciences          [All parents and Echo]
mkdir: created directory ‘drew’
mkdir: created directory ‘drew/gened’
mkdir: created directory ‘drew/gened/sciences’
```

Separating different items using space, but how about space as a char.?

- To type ‘ ’ (space) in a dir./file name, we use ‘\ ’ (backslash-space); e.g.,

```
~$ mkdir -v hello\ there
mkdir: created directory ‘hello there’
~$ ls
bunch of things ‘hello there’ bunch of things
```

File System Commands

Regular Expression can be its own topic.

- *grep [options] RE file*: **G**lobal search of **R**egular **Ex**pression in a file.

(Also has input mode)

~$ nano hamlet.txt

*copy and paster from here and save.

~$ grep king hamlet.txt

lines containing strings with ‘king’ as a substring

~$ grep ki *

all files (in dir.) and lines containing strings with ‘ki’ as a substring

~$ grep -i -e ki –e ma hamlet.txt

lines containing strings with ‘ki’ **or** ‘ma’ as a substring case **In**sensitive

Remarks

- Sometimes, admin. permission is required to perform tasks.
- *sudo [options] <command>:* **S**uper **U**ser **DO** that grants admin permission upon entering password.

  [Don't try in CoCalc!]

  sudo apt install gcc [Using the APT tool to install gcc.]

- Recall *ls –l* show permissions:

d	rwx	rwx	rwx
file/dir.	read/write/execute		
	admin	"group"	others
-	rwx	rw-	---

DREW

Summary

- Directory Hierarchy
- Commands
  - File: pwd, ls, cd, mkdir, rmdir, cp, mv, rm
  - Searching: grep
  - System: sudo, control+c
- To learn more about a command, open the manual

```
~$ man grep
 *opens manual for grep command
 *space/enter to continue; press q to quit.
~$ man sudo
```

DREW

Programming Commands – Text Editing

- *touch [options] <file>*: Create a text file and stay in current screen.

- *nano [file]*: Create a text file and enter text editor screen.

  ~$ nano hello.java

  control+O: Write Out (Save)

  control+X: Exit

  control+K: Cute Text (and store in clip-board)

  control+U: Paster Text

- Other editors you can learn: (Be careful: you may NOT know how to quit the editor)

  Atom, Vi/Vim, and Emacs are some of the popular ones.

Programming Commands – Text Editing

- *cat <file>*: Quickly view the content of a file. (Actually, for con**cat**enating files.)

```
~$ cat hello.java
public class hello                          [Note: No project/package info.]
{
        public static void main(String[] args)
        {
                System.out.println("Hello World!");
        }
}
~$
```

Programming Commands – Text Editing

- *echo [options] ["String"]*: Echoes a string to the command line.

```
~$ echo "Hello there!"
Hello there!
~$ echo
                                        [Echoing a newline.]
~$
```

- Redirecting(overriding) > output to a file/Appending >> output to a file:

  Command > file: ~$ grep king hamlet.txt > royals.txt

  Command >> file: ~$ grep madam hamlet.txt >> royals.txt

Programming Commands – Text Editing

- Redirecting/appending by Echoing:

```
~$ echo '#include <stdio.h>' > test.c                    [Redirecting into a new file.]
~$ echo 'int main() {' >> test.c                         [Appending to that file.]
~$ echo -e '\tprintf("Hello World!\n");' >> test.c       [Interpret \t as a tab.]
~$ echo -e 'return 0;\n}' >> test.c
~$ cat test.c
#include <stdio.h>
int main() {
        printf("Hello World!");
        return 0;
}
```

System Commands

- Piping | output of a command as input of a command: Command 1 | Command 2 | …

```
~$ grep king hamlet.txt                 [Sentences with “king” as substring.]
~$ grep king hamlet.txt | sort          [Those sentences sorted.]
~$ grep king hamlet.txt | sort | head -4    [The first 4 of the sorted.]
~$ grep king hamlet.txt | sort | tail -3    [The last 3 of the sorted.]

~$ cat hello.java | grep public         [Use the echoed as input to grep.]
```

Programming Commands – Compiling and Running

- Compiling and Running Java: *javac <java file>* and *java <class file>*

```
~$ javac hello.java                [Generating class file.]
~$ java hello                      [If no compilation error, run.]
Hello
~$
```

- Compiling and Running C: *gcc [options] <C file>* (GNU C Compiler) and *./<build file>*

```
 ~$ gcc test.c                     [Generating executable/build file.]
~$ ./a.out                         [If no compilation error, run.]

~$ gcc –o mybuild test.c
~$ ./mybuild
```

Summary

- Commands
  - Text Editing: touch, nano, cat, echo, > (redirect), >> (append)
  - System : |(pipe)
  - Programming: javac, java, gcc, ./ (execute)